

Collaborators: Adithya Anil Kumar, Kevin Tran, Oliver MacClure

https://github.com/adithya-blip111/data_ai_project

Task 1: Problem Statement and Dataset Selection

This project investigates the prediction of whether a student enrolled in an online course will successfully complete the course or not complete it. Over 50% of students have not completed their course across the 30 thousand records available to us. This number is a significant issue, because high fail rates indicate many students are not learning the information with how it is being taught.

Online courses can have students who disengage or fail to complete their studies. Predicting course completion can help education providers identify patterns linked to non-completion and recognize students who may need additional support. This could improve student engagement, retention, and overall course success. The main stakeholders are students, instructors, course administrators, and online education providers. Students may benefit from earlier support, while instructors and administrators can use the results to better understand student engagement and identify factors linked to course completion.

The selected dataset is the 'Student Course Completion Prediction Dataset' from Kaggle. It is publicly accessible and contains student demographic, engagement, course, and performance information that can be used to analyse and predict course completion. It is suitable because it directly supports the project's binary classification objective.

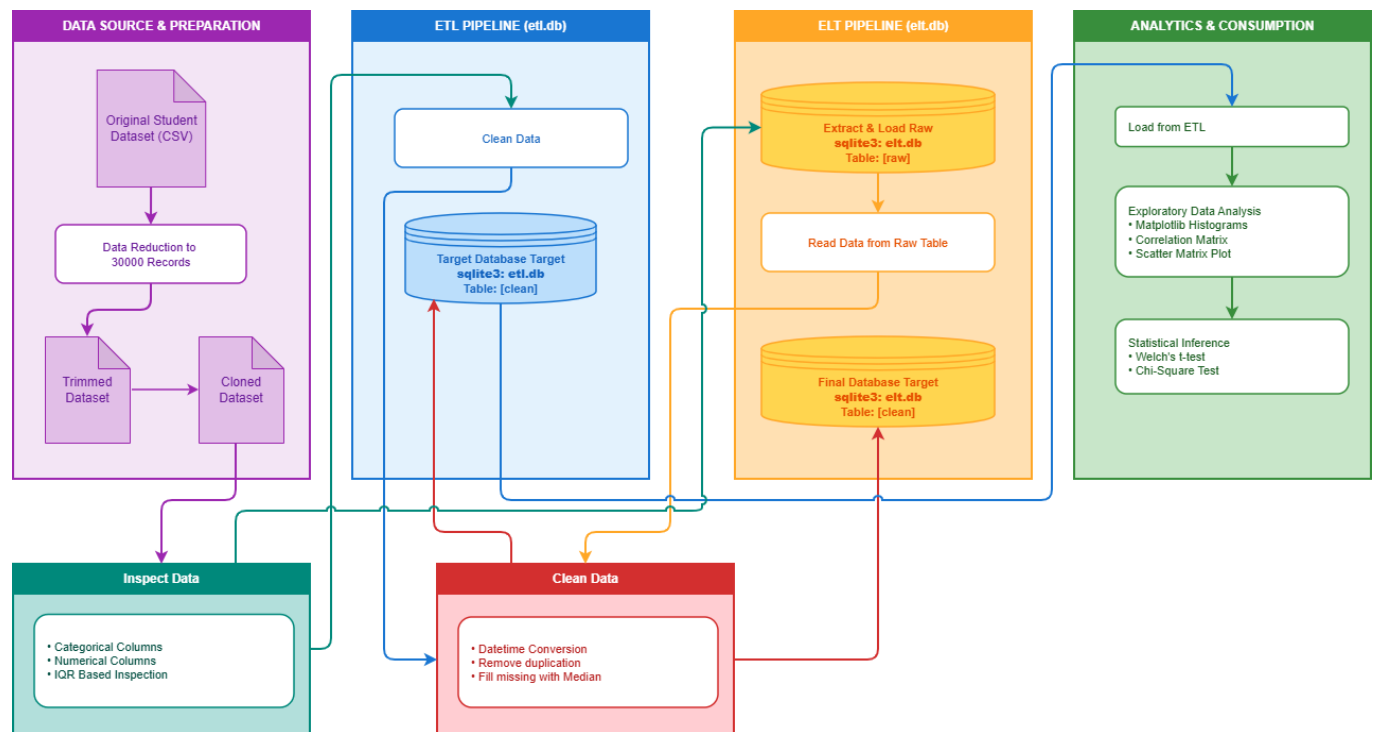
The dataset meets the requirement because there are no images included in our dataset. Our dataset is tabular, and all values are alpha-numeric.

The dataset meets the requirement because it contains 40 columns in total. With 'completed' used as the target variable, this leaves up to 39 input features, which is more than 20 and fewer than 300 as required.

The dataset was obtained from Kaggle and is titled Student Course Completion Prediction Dataset, created by Nisarg Patel. The dataset is provided under the attribution 4.0 International (CC BY 4.0) licence. Relevant documentation is available on the Kaggle Data Card, which describes the dataset, feature categories, target variables, and intended uses. The access data is 11 August 2026.

The selected 'Student Course Completion Prediction Dataset' and the proposed course completion prediction problem were submitted to the course lecturers and approved before detailed project development began.

Data Pipeline Diagram



Task 2: Data Acquisition, Inspection, Documentation

The data-acquisition process is reproducible because the notebook contains the original CSV, trimming procedure, file paths and python code. Running the notebook from beginning to end recreates the same 30000 recording work dataset and logical procedures.

Representative screenshots are included in this section to demonstrate the main inspection steps, while the complete set of data-quality checks is retained in the submitted Jupyter notebook.

```
# Run this cell for creating a trimmed version of the dataset for testing purposes.
# The trimmed dataset will contain only the first 30,000 rows of the original dataset.
import pandas as pd

df = pd.read_csv("../data/student-course-completion-prediction-dataset-original.csv")

trimmed_data = df.iloc[:30000].copy().reset_index(drop=True)

trimmed_data.to_csv(
    "../data/student-course-completion-prediction-dataset-trimmed.csv", index=False
)
print("Trimmed data created successfully.")
```

Load the dataset

```
# Load the dataset and create a copy of it for further analysis
import pandas as pd

df = pd.read_csv("../data/student-course-completion-prediction-dataset-trimmed.csv")

df_copy = df.copy(deep=True)
```

Acquisition/loading

- The original CSV was loaded into python using Pandas and `pd.read_csv()`.
- The first 30000 records were selected using `iloc[:30000]` and saved as a separate trimmed dataset for the project.
- The trimmed CSV was then loaded again and copied using `df.copy(deep=True)` to create an independent working dataframe.
- This was needed so the dataset could be inspected and processed consistently while keeping the original source data unchanged.

```
Display shape of the dataset

print("Column names and data types:")

for column_name, data_type in df_copy.dtypes.items():
    print(f"{column_name}: {data_type}")

print("\nTotal rows:", df_copy.shape[0])
print("Total columns:", df_copy.shape[1])

... Column names and data types:
Student_ID: str
Name: str
Gender: str
Age: int64
Education_Level: str
Employment_Status: str
City: str
Device_Type: str
Internet_Connection_Quality: str
Course_ID: str
Course_Name: str
Category: str
Course_Level: str
Course_Duration_Days: int64
Instructor_Rating: float64
Login_Frequency: int64
Average_Session_Duration_Min: int64
Video_Completion_Rate: float64
Discussion_Participation: int64
Time_Spent_Hours: float64
Days_Since_Last_Login: int64
Notifications_Checked: int64
Peer_Interaction_Score: float64
Assignments_Submitted: int64
...
Completed: str

Total rows: 30000
Total columns: 40
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.
```

Dataset structure

- The code loops through `df_copy.dtypes.items()` to display the name and data type of every variable.
- `df_copy.shape` was used to report the total number of records and variables.
- The output confirmed that the working dataset contained 30000 records and 40 variables.
- This check was needed to understand the dataset structure and identify which variables were numerical or categorical before carrying out the later inspection steps.

```

# Create descriptions for each variable in the dataset
# This documents what each of the 40 columns represents

variable_meanings = {
    "Student_ID": "Unique identifier assigned to each student",
    "Name": "Student name",
    "Gender": "Student gender",
    "Age": "Student age in years",
    "Education_Level": "Highest education level of the student",
    "Employment_Status": "Current employment status of the student",
    "City": "City where the student is located",
    "Device_Type": "Type of device used to access the course",
    "Internet_Connection_Quality": "Quality of the student's internet connection",
    "Course_ID": "Unique identifier for the course",
    "Course_Name": "Name of the enrolled course",
    "Category": "Subject category of the course",
    "Course_Level": "Difficulty or level of the course",
    "Course_Duration_Days": "Duration of the course in days",
    "Instructor_Rating": "Rating given to the course instructor",
    "Login_Frequency": "Frequency of student logins",
    "Average_Session_Duration_Min": "Average learning-session duration in minutes",
    "Video_Completion_Rate": "Percentage of course videos completed",
    "Discussion_Participation": "Level/count of participation in course discussions",
    "Time_Spent_Hours": "Amount of time spent on the course in hours",
    "Days_Since_Last_Login": "Number of days since the student's last login",
    "Notifications_Checked": "Number of course notifications checked",
    "Peer_Interaction_Score": "Score representing interaction with other students",
    "Assignments_Submitted": "Number of assignments submitted",
    "Assignments_Missed": "Number of assignments missed",
    "Quiz_Attempts": "Number of quiz attempts",
    "Quiz_Score_Avg": "Average quiz score",
    "Project_Grade": "Grade received for the course project",
    "Progress_Percentage": "Percentage of overall course progress",
    "Rewatch_Count": "Number of times course material was rewatched",
    "Payment_Mode": "Method used to pay for or access the course",
    "Fee_Paid": "Indicates whether the student paid the course fee",
    "Payment_Amount": "Recorded payment/course amount",
    "Discount_Used": "Indicates whether a discount was used",
    "App_Usage_Percentage": "Percentage representing use of the learning app",
    "Enrollment_Date": "Date the student enrolled in the course",
    "Reminder_Emails_Clicked": "Number of reminder emails clicked",
    "Support_Tickets_Raised": "Number of support requests raised",
    "Satisfaction_Rating": "Student satisfaction rating",
    "Completed": "Target variable showing whether the course was completed",
}

# Create a data dictionary containing:
# column name, data type and variable meaning
data_dictionary = pd.DataFrame(
    [
        {
            "Variable": df_copy.columns,
            "Data_Type": [str(df_copy[col].dtype) for col in df_copy.columns],
            "Meaning": [variable_meanings[col] for col in df_copy.columns],
        }
    ]
)

# Display the complete data dictionary
print(data_dictionary.to_string(index=False))

```

Variable	Data_Type	Meaning
Student_ID	str	Unique identifier assigned to each student
Name	str	Student name
Gender	str	Student gender
Age	int64	Student age in years
Education_Level	str	Highest education level of the student
Employment_Status	str	Current employment status of the student
City	str	City where the student is located
Device_Type	str	Type of device used to access the course
Internet_Connection_Quality	str	Quality of the student's internet connection
Course_ID	str	Unique identifier for the course
Course_Name	str	Name of the enrolled course
Category	str	Subject category of the course
Course_Level	str	Difficulty or level of the course
Course_Duration_Days	int64	Duration of the course in days
Instructor_Rating	float64	Rating given to the course instructor
Login_Frequency	int64	Frequency of student logins
Average_Session_Duration_Min	int64	Average learning-session duration in minutes
Video_Completion_Rate	float64	Percentage of course videos completed
Discussion_Participation	int64	Level/count of participation in course discussions
Time_Spent_Hours	float64	Amount of time spent on the course in hours
Days_Since_Last_Login	int64	Number of days since the student's last login
Notifications_Checked	int64	Number of course notifications checked
Peer_Interaction_Score	float64	Score representing interaction with other students
Assignments_Submitted	int64	Number of assignments submitted
Reminder_Emails_Clicked	int64	Number of reminder emails clicked
Support_Tickets_Raised	int64	Number of support requests raised
Satisfaction_Rating	float64	Student satisfaction rating
Completed	str	Target variable showing whether the course was completed

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings.

Data Dictionary

- The code created a description for each of the 40 variables in the dataset.
- The variable name, data type and definition were combined into a structured data dictionary.

- This was needed to ensure that each variable was interpreted correctly and consistently before the data-quality checks were carried out.

```
# Display duplicate Student_ID records
duplicate_student_rows = df_copy[df_copy["Student_ID"].duplicated(keep=False)]

print("Duplicate records with the same Student_ID:")
print(duplicate_student_rows.to_string(index=False))

# Display completely identical duplicate records
complete_duplicate_rows = df_copy[df_copy.duplicated(keep=False)]

print("\nComplete duplicate records:")
print(complete_duplicate_rows.to_string(index=False))
```

✓ 0.0s

Duplicate records with the same Student_ID:
Empty DataFrame
Columns: [Student_ID, Name, Gender, Age, Education_Level, Employment_Status, City, Device_Type, Internet_Connection_Quality, Course_ID, Course_Name, Category, Index: []

Complete duplicate records:
Empty DataFrame
Columns: [Student_ID, Name, Gender, Age, Education_Level, Employment_Status, City, Device_Type, Internet_Connection_Quality, Course_ID, Course_Name, Category, Index: []

Duplicate-record inspection

- The code first used `Student_ID.duplicated(keep=False)` to identify any student IDs that appeared more than once.
- `df_copy.duplicated(keep=False)` was then used to check for completely identical rows across all 40 variables
- Both checks returned empty DataFrames, which means no duplicate student IDs or fully duplicated records were found
- This check was needed to confirm that repeated observations would not distort the later analysis or cleaning process.

```
# Check columns with missing values
missing = df_copy.isna().sum()
missing_columns = missing[missing > 0].sort_values(ascending=False)

print("Columns containing missing values:")

if missing_columns.empty:
    print("No missing values found.")
else:
    for column_name, missing_count in missing_columns.items():
        print(f"{column_name}: {missing_count}")

# Display rows containing missing values
rows_with_missing = df_copy[df_copy.isna().any(axis=1)]

print("\nRows containing missing values:")
print(rows_with_missing.to_string(index=False))
```

✓ 0.0s

Columns containing missing values:
No missing values found.

Missing-value inspection

- The code used `isna().sum()` to count missing values in every column.
- It filtered the results to display only columns containing missing data.
- The output showed that no missing values were found in the working dataset.
- This checked confirmed that no missing observations required fixing during the cleaning stage.

[illegible]

```
# Check categorical columns for hidden inconsistencies
# Example: "Male", "male", or " Male " would be treated as possible duplicates
inconsistency_found = False

for col in category_check_cols:
    original_count = df_copy[col].nunique(dropna=False)

    cleaned_count = (
        df_copy[col].astype("string").str.strip().str.lower().nunique(dropna=False)
    )

    if original_count != cleaned_count:
        inconsistency_found = True
        print(
            col,
            "- possible inconsistent categories:",
            original_count,
            "original vs",
            cleaned_count,
            "standardised",
        )

if not inconsistency_found:
    print("No inconsistencies found.")
```

✓ 0.1s

No inconsistencies found.

Inspection of categorical values

- The code first identifies columns containing categorical/text data
- A selected list of categorical variables is then created for inspection
- Each column is looped through, and its unique values are displayed, excluding missing entries.
- This check helps to identify unexpected labels, spelling differences, or inconsistent categories that may require cleaning.
- The categories were also standardised temporarily by removing surrounding spaces and converting text to lowercase, allowing hidden formatting inconsistencies to be detected.
- The output showed no hidden formatting inconsistencies. The categorical values were already consistently formatted.

```
# Find all numerical columns in the raw dataset
numerical_cols = df_copy.select_dtypes(include="number").columns

# Check the minimum value and number of negative values in each numerical column
for col in numerical_cols:
    minimum_value = df_copy[col].min()
    negative_count = (df_copy[col] < 0).sum()

    print(
        f"{col}: " f"minimum = {minimum_value}, " f"negative values = {negative_count}"
    )
```

(15) ✓ 0.0s

```
... Age: minimum = 17, negative values = 0
Course_Duration_Days: minimum = 25, negative values = 0
Instructor_Rating: minimum = 4.1, negative values = 0
Login_Frequency: minimum = 0, negative values = 0
Average_Session_Duration_Min: minimum = 5, negative values = 0
Video_Completion_Rate: minimum = 5.0, negative values = 0
Discussion_Participation: minimum = 0, negative values = 0
Time_Spent_Hours: minimum = 0.5, negative values = 0
Days_Since_Last_Login: minimum = 0, negative values = 0
Notifications_Checked: minimum = 0, negative values = 0
Peer_Interaction_Score: minimum = 0.0, negative values = 0
Assignments_Submitted: minimum = 0, negative values = 0
Assignments_Missed: minimum = 0, negative values = 0
Quiz_Attempts: minimum = 0, negative values = 0
Quiz_Score_Avg: minimum = 19.6, negative values = 0
Project_Grade: minimum = 6.8, negative values = 0
Progress_Percentage: minimum = 9.7, negative values = 0
Rewatch_Count: minimum = 0, negative values = 0
Payment_Amount: minimum = 0, negative values = 0
App_Usage_Percentage: minimum = 0, negative values = 0
Reminder_Emails_Clicked: minimum = 0, negative values = 0
Support_Tickets_Raised: minimum = 0, negative values = 0
Satisfaction_Rating: minimum = 1.0, negative values = 0
```



```

# Find all numerical columns in the raw dataset
numerical_cols = df_copy.select_dtypes(include="number").columns

# Check the minimum value and number of negative values in each numerical column
for col in numerical_cols:
    minimum_value = df_copy[col].min()
    negative_count = (df_copy[col] < 0).sum()

    print(
        f"{col}: minimum = {minimum_value}, negative values = {negative_count}"
    )

```

[15] ✓ 0.0s

```

Age: minimum = 17, negative values = 0
Course_Duration_Days: minimum = 25, negative values = 0
Instructor_Rating: minimum = 4.1, negative values = 0
Login_Frequency: minimum = 0, negative values = 0
Average_Session_Duration_Min: minimum = 5, negative values = 0
Video_Completion_Rate: minimum = 5.0, negative values = 0
Discussion_Participation: minimum = 0, negative values = 0
Time_Spent_Hours: minimum = 0.5, negative values = 0
Days_Since_Last_Login: minimum = 0, negative values = 0
Notifications_Checked: minimum = 0, negative values = 0
Peer_Interaction_Score: minimum = 0.0, negative values = 0
Assignments_Submitted: minimum = 0, negative values = 0
Assignments_Missed: minimum = 0, negative values = 0
Quiz_Attempts: minimum = 0, negative values = 0
Quiz_Score_Avg: minimum = 19.6, negative values = 0
Project_Grade: minimum = 6.8, negative values = 0
Progress_Percentage: minimum = 9.7, negative values = 0
Rewatch_Count: minimum = 0, negative values = 0
Payment_Amount: minimum = 0, negative values = 0
App_Usage_Percentage: minimum = 0, negative values = 0
Reminder_Emails_Clicked: minimum = 0, negative values = 0
Support_Tickets_Raised: minimum = 0, negative values = 0
Satisfaction_Rating: minimum = 1.0, negative values = 0

```

```

# Check the minimum and maximum values for the rating columns
# Both ratings should stay between their valid limits and must not exceed 5

```

```

for col in ["Instructor_Rating", "Satisfaction_Rating"]:
    print(
        f"{col}: "
        f"minimum = {df_copy[col].min()}, "
        f"maximum = {df_copy[col].max()}, "
        f"values above 5 = {(df_copy[col] > 5).sum()}"
    )

```

[16] ✓ 0.0s

```

Instructor_Rating: minimum = 4.1, maximum = 4.7, values above 5 = 0
Satisfaction_Rating: minimum = 1.0, maximum = 5.0, values above 5 = 0

```

```

# List the columns that represent percentages/rates

```

```

percentage_cols = [
    "Video_Completion_Rate",
    "Progress_Percentage",
    "App_Usage_Percentage",
]

```

```

# Check the minimum, maximum, and how many values exceed 100

```

```

for col in percentage_cols:
    print(
        f"{col}: "
        f"minimum = {df_copy[col].min()}, "
        f"maximum = {df_copy[col].max()}, "
        f"values above 100 = {(df_copy[col] > 100).sum()}"
    )

```

[17] ✓ 0.0s

```

Video_Completion_Rate: minimum = 5.0, maximum = 99.9, values above 100 = 0
Progress_Percentage: minimum = 9.7, maximum = 97.4, values above 100 = 0
App_Usage_Percentage: minimum = 0, maximum = 100, values above 100 = 0

```

Numerical validity, value ranges and percentage ranges

- The first code block identifies all numerical columns, then checks the minimum value and counts any negative values in each one.
- The output shows that no negative values were found in the numerical variables.
- The second code block checks the valid range of Instructor_Rating and Satisfaction_Rating, including whether any values exceed 5
- The output shows both rating variables remained within the expected 0-5 range.
- These checks were needed to identify impossible or invalid numerical values before the cleaning stage.
- The final check inspected Video_Completion_Rate, Progress_Percentage, and App_Usage_Percentage to confirm that percentage values remained within the expected 0-100% range.
- The output showed that no values exceeded 100% in any of the three variables.

```
# Convert Enrollment_Date temporarily for inspection only
# dayfirst=True because the dataset uses DD/MM/YYYY
enrollment_date_check = pd.to_datetime(
    df_copy["Enrollment_Date"], dayfirst=True, errors="coerce"
)

# Count dates that could not be converted
print("Invalid Enrollment_Date values:", enrollment_date_check.isna().sum())

# Show the earliest and latest enrollment dates
print("Earliest enrollment date:", enrollment_date_check.min())
print("Latest enrollment date:", enrollment_date_check.max())
```

[22] ✓ 0.0s

```
Invalid Enrollment_Date values: 0
Earliest enrollment date: 2023-10-17 00:00:00
Latest enrollment date: 2025-10-06 00:00:00
```

Enrolment-date validity inspection

- The code temporarily converted Enrollment_Data to a datetime format using - pd.to.datetime() with dayfirst=True, based on the assumed DD/MM/YYYY format. errors="coerce" was used so any valid dates would become missing values and could be counted.
- The output showed 0 invalid dates, with enrolment dates ranging from 17 October 2023 to 6 October 2025
- This check confirms that the date values could be interpreted consistently before cleaning and analysis.

```

# Find the outliers in the numerical columns using the IQR method
numerical_cols = df_copy.select_dtypes(include="number").columns

# Store the outlier results
outlier_summary = []

for col in numerical_cols:

    # Calculate the first and third quartiles
    q1 = df_copy[col].quantile(0.25)
    q3 = df_copy[col].quantile(0.75)

    # Calculate the interquartile range
    iqr = q3 - q1

    # Define the usual IQR outlier limits
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr

    # Count values outside the limits
    outlier_count = ((df_copy[col] < lower_bound) | (df_copy[col] > upper_bound)).sum()

    # Save the result
    outlier_summary.append(
        {
            "Column": col,
            "Lower_Bound": round(lower_bound, 2),
            "Upper_Bound": round(upper_bound, 2),
            "Outlier_Count": outlier_count,
        }
    )

# Convert the results into a dataframe
outlier_summary = pd.DataFrame(outlier_summary)

# Show only columns where potential outliers were found
print(outlier_summary[outlier_summary["Outlier_Count"] > 0].to_string(index=False))

```

28 | ✓ 0.0s

	Column	Lower_Bound	Upper_Bound	Outlier_Count
	Age	7.50	43.50	32
	Login_Frequency	-1.50	10.50	114
	Average_Session_Duration_Min	6.00	62.00	183
	Discussion_Participation	-2.00	6.00	408
	Time_Spent_Hours	-8.20	15.00	309
	Days_Since_Last_Login	-11.00	21.00	1199
	Notifications_Checked	-0.50	11.50	320
	Peer_Interaction_Score	0.70	11.90	101
	Assignments_Submitted	1.00	9.00	118
	Assignments_Missed	1.00	9.00	129
	Quiz_Attempts	-2.50	9.50	223
	Quiz_Score_Avg	38.65	107.85	84
	Project_Grade	26.05	110.45	111
	Progress_Percentage	20.15	87.75	167
	Rewatch_Count	-2.00	6.00	420
	App_Usage_Percentage	14.50	122.50	119
	Reminder_Emails_Clicked	-2.00	6.00	377
	Support_Tickets_Raised	-1.50	2.50	1838
	Satisfaction_Rating	2.05	6.45	110

IQR-Based outlier inspection (measures the spread of the middle 50% of the data)

- The code selected all numerical variables and calculated the first quartile (Q1), third quartile (Q3), and interquartile range (IQR) for each column.
- Lower and upper limits were calculated using $1.5 \times \text{IQR}$, and values outside these limits were counted as potential outliers.
- The output showed that several numerical variables contained potential outliers, with the number varying between columns.
- These values were flagged for review rather than automatically treated as errors, because an unusual value can still be valid.

```

# Final audit of every column in the untouched dataset
# This checks:
# - data type
# - missing values
# - number of unique values
# - minimum and maximum values for numerical columns

audit_rows = []

for col in df_copy.columns:

    # Basic information for every column
    column_info = {
        "Column": col,
        "Data_Type": str(df_copy[col].dtype),
        "Missing": df_copy[col].isna().sum(),
        "Unique": df_copy[col].nunique(dropna=False),
    }

    # Add minimum and maximum only if the column is numerical
    if pd.api.types.is_numeric_dtype(df_copy[col]):
        column_info["Min"] = df_copy[col].min()
        column_info["Max"] = df_copy[col].max()
        column_info["Mean"] = df_copy[col].mean()
        column_info["Median"] = df_copy[col].median()
    else:
        column_info["Min"] = "-"
        column_info["Max"] = "-"
        column_info["Mean"] = "-"
        column_info["Median"] = "-"

    audit_rows.append(column_info)

# Turn the audit results into a dataframe
final_audit = pd.DataFrame(audit_rows)

# Allow pandas to display every row in the audit table
pd.set_option("display.max_rows", None)

# Show the complete final audit
print(final_audit.to_string(index=False))

```

0.0s

	Column	Data_Type	Missing	Unique	Min	Max	Mean	Median
	Student_ID	str	0	30000	-	-	-	-
	Name	str	0	300	-	-	-	-
	Gender	str	0	3	-	-	-	-
	Age	int64	0	33	17	49	25.783533	26.0
	Education_Level	str	0	5	-	-	-	-
	Employment_Status	str	0	4	-	-	-	-
	City	str	0	15	-	-	-	-
	Device_Type	str	0	3	-	-	-	-
	Internet_Connection_Quality	str	0	3	-	-	-	-
	Course_ID	str	0	8	-	-	-	-
	Course_Name	str	0	8	-	-	-	-
	Category	str	0	5	-	-	-	-
	Course_Level	str	0	3	-	-	-	-
	Course_Duration_Days	int64	0	8	25	90	51.779333	45.0
	Instructor_Rating	float64	0	7	4.1	4.7	4.44347	4.5
	Login_Frequency	int64	0	16	0	15	4.781833	5.0
	Average_Session_Duration_Min	int64	0	72	5	76	33.880967	34.0
	Video_Completion_Rate	float64	0	929	5.0	99.9	62.275667	64.1
	Discussion_Participation	int64	0	13	0	12	2.3286	2.0
	Time_Spent_Hours	float64	0	209	0.5	25.6	3.886817	2.7
	Days_Since_Last_Login	int64	0	71	0	99	6.1358	4.0
	Notifications_Checked	int64	0	19	0	18	5.221933	5.0
	Peer_Interaction_Score	float64	0	101	0.0	10.0	6.297417	6.3
	Assignments_Submitted	int64	0	11	0	10	4.7761	5.0
...	Reminder_Emails_Clicked	int64	0	13	0	13	2.3436	2.0
	Support_Tickets_Raised	int64	0	8	0	7	0.867267	1.0
	Satisfaction_Rating	float64	0	39	1.0	5.0	4.134583	4.2
	Completed	str	0	2	-	-	-	-

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#).

Clean dataset

Final audit of every column

- The code loops through every column and records its data type, number of missing values, and number of unique values.
- For numerical variables, it also calculates min, max, mean and median.
- The results are combined into one audit table so the full dataset can be reviewed in a consistent format.

- This final check was needed to confirm that all variables and columns had been inspected before moving to the cleaning stage.

Documented Assumptions

- Enrollment_Date was interpreted using the DD/MM/YYYY format.
- Percentage/rate variables were expected to remain within 0-100%.
- Instructor_Rating and Satisfaction_Rating were expected to remain within 0-5.
- Completed was expected to contain only Completed or Not Completed.
- Fee_Paid and Discount_Used were expected to contain only Yes or NO.
- Values identified by the IQR method were treated as potential outliers for review, rather than automatically being considered invalid or removed.

Task 3: Data Cleansing and Transformation

```
def clean_data(data):
    # Create a copy so the raw dataset stays unchanged
    clean = data.copy(deep=True)

    # Convert Enrollment_Date to a proper datetime type
    clean["Enrollment_Date"] = pd.to_datetime(
        clean["Enrollment_Date"], dayfirst=True, errors="raise"
    )

    # Fill median values for missing values in numerical columns
    for col in [
        "Instructor_Rating",
        "Time_Spent_Hours",
        "Peer_Interaction_Score",
        "Quiz_Score_Avg",
        "Project_Grade",
        "Progress_Percentage",
        "App_Usage_Percentage",
        "Satisfaction_Rating",
    ]:
        clean[col] = clean[col].fillna(clean[col].median())

    # Remove completely identical duplicate rows, if any
    clean = clean.drop_duplicates()

    # Create completion-status column used by later graphs
    clean["Completed_Status"] = (clean["Completed"] == "Completed").astype(bool)

    # Reset the row numbers
    clean = clean.reset_index(drop=True)

    return clean
```

- Use `clean = data.copy(deep=True)` to create an independent dataset copy, so that the original raw data remains untouched from unintended side effects from having it used elsewhere in the program.
- The `pd.to_datetime` function helps transform the column ("Enrollment_Date") into pandas' datetime object. `dayfirst=True` should be set to resolve date ambiguity (e.g., reading 05/02 as Feb 5th instead of May 2nd), and `errors="raise"` to flag invalid dates by breaking the code.
- For loop iterates through the listed numerical columns to replace empty cells with its respective median value; chosen over the average to preserve the typical user data's integrity, because less resistance to outliers degrades it.

- To prevent double counting from occurring during later analysis, `clean.drop_duplicates()` scans rows and removes any that are identical across every column.
- This code line generates a new column (“Completed_Status”) by checking if the string “Completed” aligns with the text in the “Completed” column, alongside `.astype(bool)` tacked on to turn the result into True or False. (faster visualizations, less memory).
- Original row numbers (the index) will have missing gaps from duplicate drops removing rows from the dataset, and `clean.reset_index(drop=True)` rennumbers everything from zero.

```
# Clean the dataset and store the cleaned version in a new variable
df_clean_preview = clean_data(df_copy)
```

- The previously defined function is called by this to apply all cleaning steps to the raw data (`df_copy`) and assigning the results in the new variable `df_clean_preview`.

```
# Check that the duplicate was removed
print("Duplicate rows:", df_clean_preview.duplicated().sum())

# Check that Student_ID values are now unique
print("Duplicate Student_IDs:", df_clean_preview["Student_ID"].duplicated().sum())

# Check that Enrollment_Date is now a datetime data type
print("Enrollment_Date type:", df_clean_preview["Enrollment_Date"].dtype)
```

✓ 0.0s

```
Duplicate rows: 0
Duplicate Student_IDs: 0
Enrollment_Date type: datetime64[us]
```

- The total number of identical rows remaining in the entire dataset is calculated by `df_clean_preview.duplicated().sum()`, where it should then be printed to indicate whether the earlier data cleaning step removed all full-row duplicates successfully.
- Specifically, within that column, student ids get isolated by `df_clean_preview["Student_ID"].duplicated().sum()`, so that only one of every student appears.
- To confirm the conversion of the date column, `df_clean_preview["Enrollment_Date"].dtype` outputs the current format, so to see if it turned into a proper pandas' datetime object, i.e. `datetime64`.


```
# Final verification of the cleaned dataset

# Check for missing values
print("Total missing values:", df_clean_preview.isna().sum().sum())

# Check age range after cleaning
print("Age range:", df_clean_preview["Age"].min(), "to", df_clean_preview["Age"].max())

# Check for any negative numerical values
numerical_cols = df_clean_preview.select_dtypes(include="number").columns
negative_total = (df_clean_preview[numerical_cols] < 0).sum().sum()

print("Total negative numerical values:", negative_total)

# Check final dataset size
print("Final dataset shape:", df_clean_preview.shape)

print("\nOriginal shape:", df_copy.shape)
print("Cleaned shape:", df_clean_preview.shape)
print("Complete duplicate rows remaining:", df_clean_preview.duplicated().sum())
print(
    "Duplicate Student IDs remaining:",
    df_clean_preview["Student_ID"].duplicated().sum(),
)
print("Rows removed:", len(df_copy) - len(df_clean_preview))

print("\nFirst five cleaned records:")
print(
    df_clean_preview[
        [
            "Student_ID",
            "Name",
            "Gender",
            "Age",
            "Education_Level",
            "Employment_Status",
            "Completed",
        ]
    ]
    .head()
    .to_string(index=False)
)

✓ 0.1s

Total missing values: 0
Age range: 17 to 49
Total negative numerical values: 0
Final dataset shape: (30000, 41)

Original shape: (30000, 40)
Cleaned shape: (30000, 41)
Complete duplicate rows remaining: 0
Duplicate Student IDs remaining: 0
Rows removed: 0

First five cleaned records:
Student_ID      Name Gender  Age Education_Level Employment_Status  Completed
STU100000  Vihaan Patel   Male   19      Diploma           Student    Completed
STU100001   Arjun Nair  Female  17      Bachelor           Student    Not Completed
STU100002  Aditya Bhardwaj Female  34      Master            Student    Completed
STU100003   Krishna Singh Female  29      Diploma           Employed    Completed
STU100004   Krishna Nair Female  19      Master            Self-Employed    Completed
```

- To confirm that the earlier data imputation step (substituted values) was successful, `df_clean_preview.isna().sum().sum()` combs the entire dataset to calculate the grand total of empty cells, returning with 0 as the output if none present.
- `.min()` and `.max()` are called on the “Age” column to verify whether the data has no erroneous outliers (e.g. age 999 or -5) that slipped through, and that it falls within a realistic range (17 to 49).
- `.select_dtypes(include="number")`, paired with the `< 0` condition, automatically isolates all numerical columns and checks for negative data values.

- `.head().to_string(index=False)` is called on the specific list of columns to omit arbitrary rows number when printing the first five rows.

```
import sqlite3

# Use the cleaned data already reviewed above
df_etl_clean = df_clean_preview.copy(deep=True)

with sqlite3.connect("../data\\etl.db") as conn:
    df_etl_clean.to_sql("clean", conn, if_exists="replace", index=False)

print("ETL completed.")
print("Cleaned table shape:", df_etl_clean.shape)
```

✓ 0.4s

```
ETL completed.
Cleaned table shape: (30000, 41)
```

- The built-in Python library is loaded by `import sqlite3` to interact with SQLite databases.
- The cleaned dataset gets a copy made from `df_etl_clean = df_clean_preview .copy(deep=True)` to prevent the previous preview variable from being altered.
- A local SQLite database file gets a connection established by `with sqlite3.connect`, where it automatically handles the closure of the connection when finished.
- The pandas dataframe gets directly written into a database table (clean) by `df_etl_clean.to sql. If_exists="replace"` overrides any old version of the table, and `index=False` prevents pandas from saving row numbers as part of the data.
- The final row and column count gets displayed in the output cell with `df_etl_clean.shape`.

```

with sqlite3.connect("../data\\elt.db") as conn:
    # Load the untouched raw data first
    df_copy.to_sql("raw", conn, if_exists="replace", index=False)
    # Read the raw table back into Pandas
    df_elt_raw = pd.read_sql("SELECT * FROM raw", conn)
    # Transform the data after loading
    df_elt_clean = clean_data(df_elt_raw)
    # Store the cleaned result as a second table
    df_elt_clean.to_sql("clean", conn, if_exists="replace", index=False)

print("ELT completed.")
print("Raw table shape:", df_elt_raw.shape)
print("Cleaned table shape:", df_elt_clean.shape)

```

✓ 1.4s

```

ELT completed.
Raw table shape: (30000, 40)
Cleaned table shape: (30000, 41)

```

- df_copy.to_sql writes the uncleaned dataset into an SQLite database table labelled “raw”
- The raw database table is read using standard SQL back into Python by df_elt_raw = pd.read_sql(“SELECT * FROM raw”, conn).
- df_elt_clean = clean_data(df_elt_raw) passes the extracted records from the database into a cleaning pipeline for the purposes of handling missing values, formatting and feature engineering.
- The dataset, after the final processing, gets written into a second database table named clean, both raw and clean are stored separately.
- Print statements to output confirmation and compare raw and clean dataset shapes to verify that feature engineering had added a new column without deleting rows.

```

with sqlite3.connect("../data\\etl.db") as conn:
    etl_tables = pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", conn)
print("Tables in etl.db:")
print(etl_tables.to_string(index=False))

with sqlite3.connect("../data\\elt.db") as conn:
    elt_tables = pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", conn)
print("\nTables in elt.db:")
print(elt_tables.to_string(index=False))

```

✓ 0.0s

Tables in etl.db:

```

name
clean

```

Tables in elt.db:

```

name
raw
clean

```

- Both the specified SQLite databases (etl.db and elt.db) are connected to.
- A query gets executed by `pd.read_sql` on the `sqlite_master` table to fetch and store all table names in each database.
- `elt_tables.to_string` is printed to output all stored table names for each of the database (hiding default row indices also).

```

# Print variable names starting with 'df' to verify the data in the current namespace
print("\nVariables starting with 'df':")

for name in list(globals()):
    if name.startswith("df"):
        print(name)

```

✓ 0.0s

Variables starting with 'df':

```

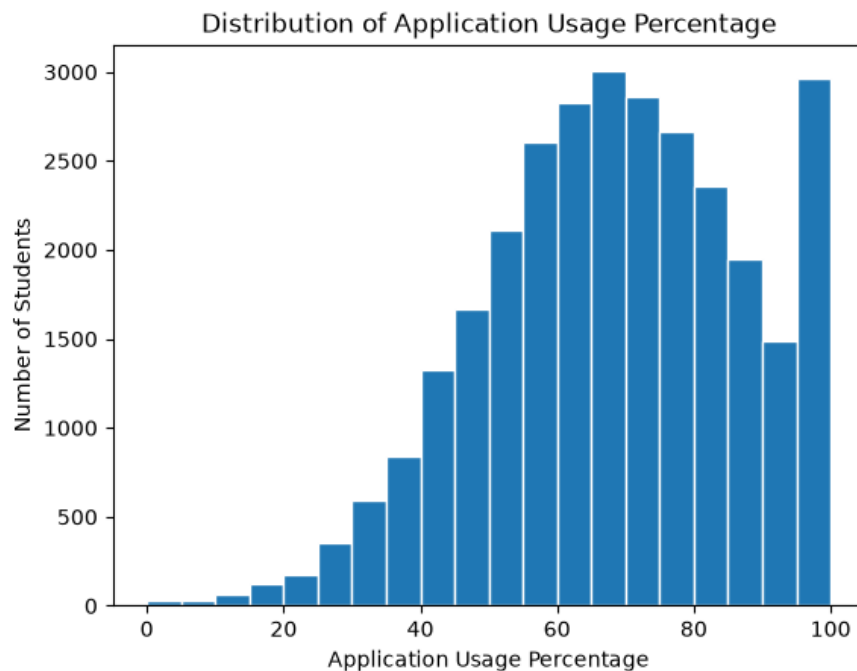
df
df_copy
df_clean_preview
df_etl_clean
df_elt_raw
df_elt_clean
df_sensitivity
df_analysis
df_clean_stats

```

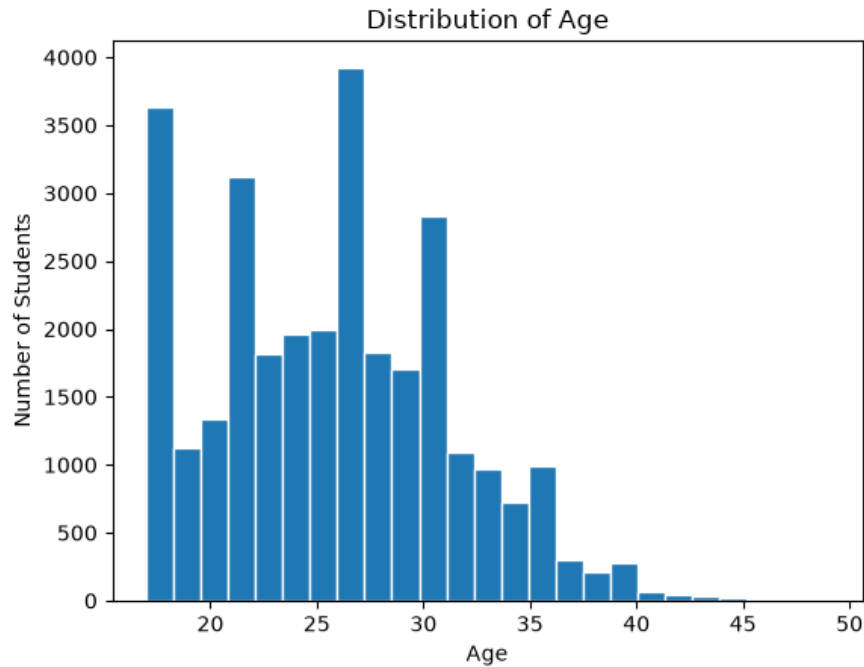
- All the active variables stored in the Python session's memory gets retrieved into a full list by `list(globals())`.

- If `name.startswith("df")`, scans and isolates all environment variables to only those with the prefix “df” (pandas DataFrames).
- The filtered list is printed with `print(name)` to see all loaded and available versions of the dataset, like raw, clean, ETL, ELT.

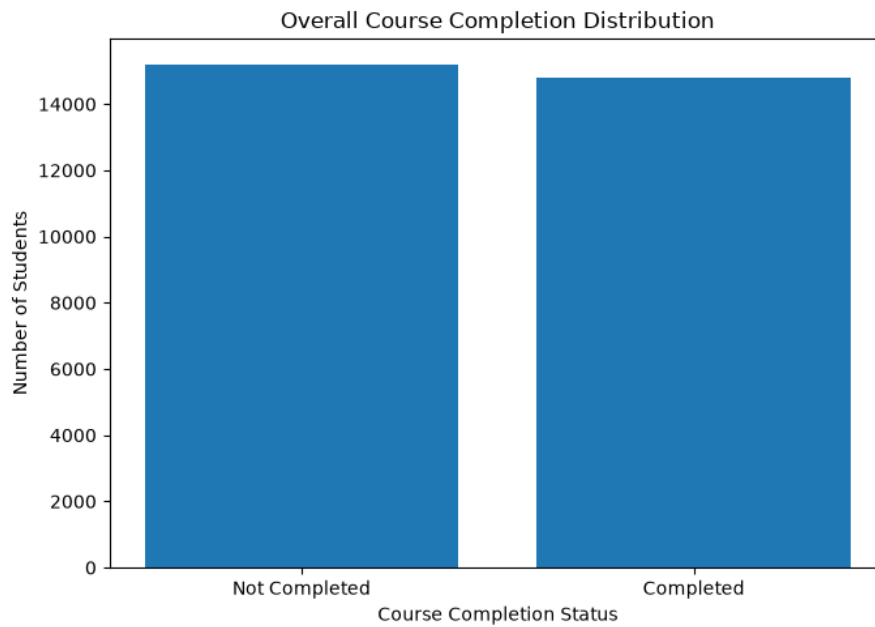
Task 4: Exploratory Data Analysis and Visualisation

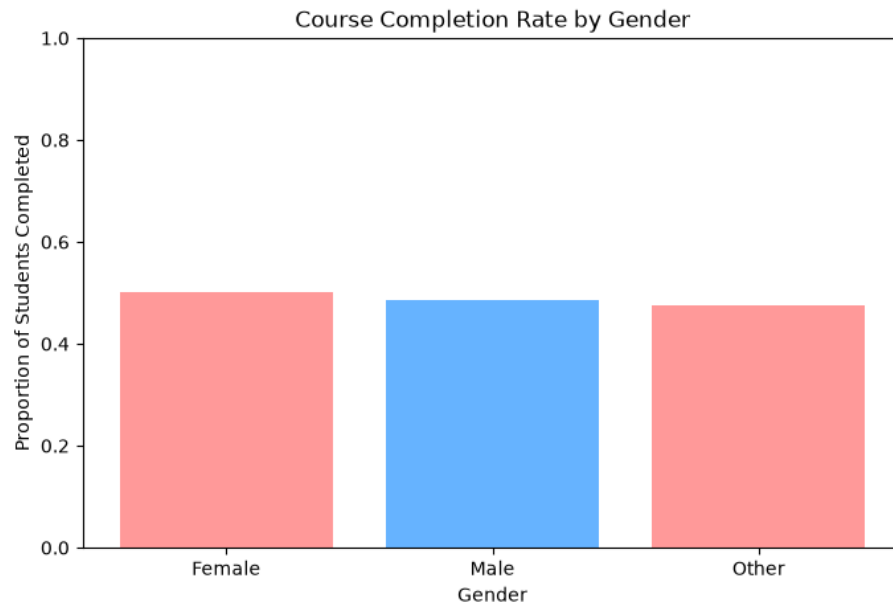


There's a strong left-skewed distribution displayed by the application usage percentage variable, which showcase that most students have usage in the moderate to high range, the bell curve peaking between 60% and 80%. It shows that most students in the dataset are more likely to have used the app significantly. There is also a large spike near to 100%.



The graph shows varied ages among the students, but most notably, younger student demographics make up a large portion, where the most peaks of the number of students per age group resides between ages 18 and 26. Although a moderate amount is seen in older ranges, like up to 30.

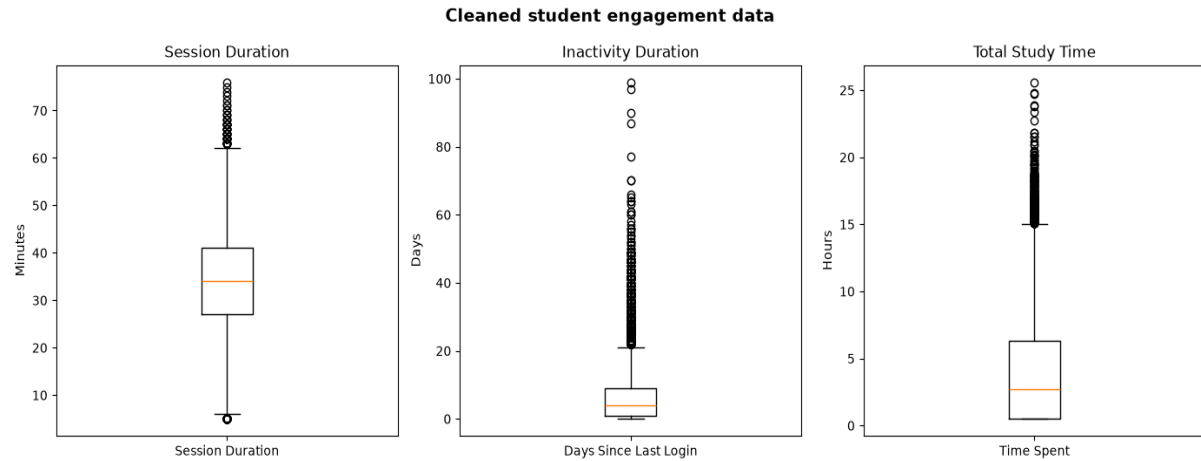




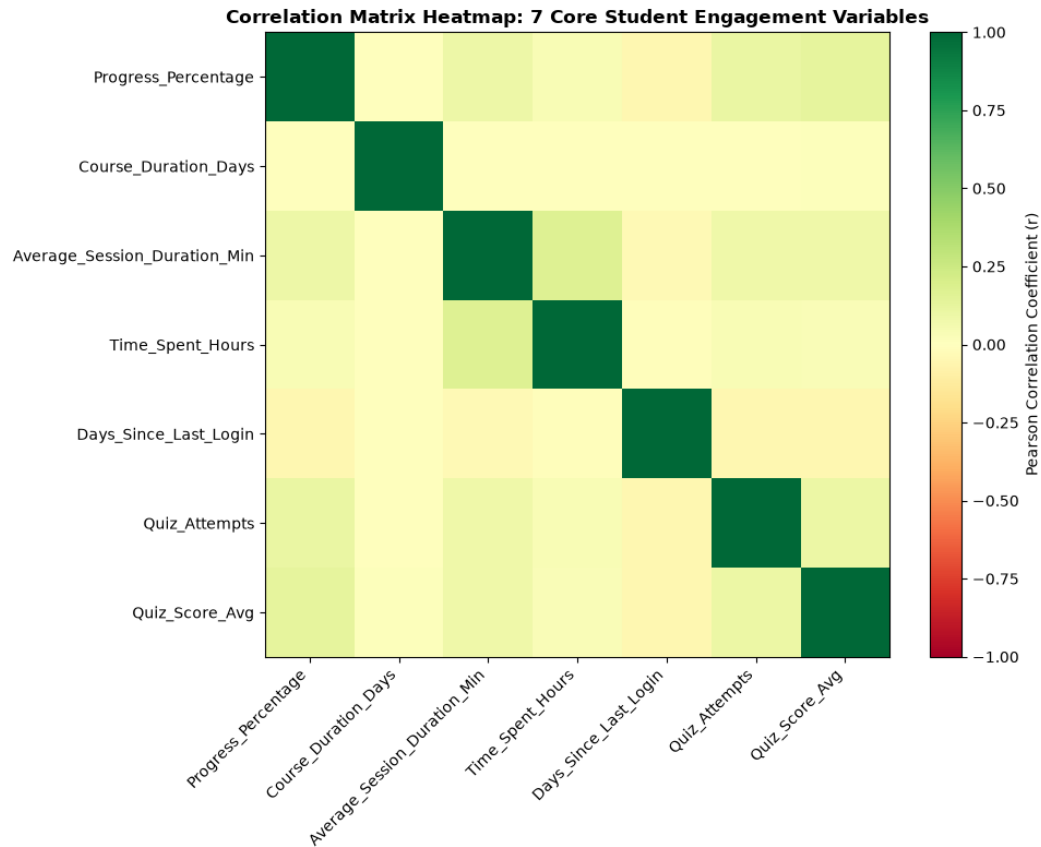
```
Course Completion Distribution:
Completed
Not Completed    15208
Completed        14792
Name: count, dtype: int64

Course Completion Rate by Gender:
Gender
Female    0.500528
Male      0.485915
Other     0.474797
Name: is_completed, dtype: float64
```

The distribution of the overall course completion is relatively split halfway, with about 15208 students having completed it, and 14792 that did not. Delving into the specific gender, it shows that females complete the course at an approximate rate of 0.5 (50%), males at a rate of 0.486 (48.6%), and the remaining (other) being slightly lower, at a rate of 0.474 (47.4%). Since the three bars are near identical, it is safe to assume gender is not a predictive factor for this dataset, meaning a student is equally likely to finish a course, regardless of their gender.



The median of session duration sits at around 35 minutes, with the interquartile range being around 12 minutes wide. Its full range however is much wider spanning from 2 minutes up to about 62 minutes, making its full range 60 minutes wide. The inactivity duration graph has a very short box and whisker. Its maximum value lies at around 20 days, but its upper quartile value lies around 10 days. Its highest outlier value sits at around 100 days putting it far above the box and whisker, leading to its short appearance. The final graph, total study time, has a median of about 3 hours. An interesting point about this graph is its lower quarter value being the same as its minimum value, this indicates a heavy group at this minimum value of zero. Although all these graphs have significant visual outliers, these values are being compared across 30,000 results. This means few hundred values that are visible are definitively outliers from the rest.



```

--- Correlation Matrix ---
Progress_Percentage  Course_Duration_Days  \
Progress_Percentage      1.00      0.00
Course_Duration_Days      0.00      1.00
Average_Session_Duration_Min  0.00      0.00
Time_Spent_Hours          0.03      0.00
Days_Since_Last_Login     -0.05      0.01
Quiz_Attempts             0.11      0.01
Quiz_Score_Avg            0.13      0.01

Average_Session_Duration_Min  Time_Spent_Hours  \
Progress_Percentage           0.09      0.03
Course_Duration_Days           0.00      0.00
Average_Session_Duration_Min     1.00      0.17
Time_Spent_Hours                 0.17      1.00
Days_Since_Last_Login          -0.03     -0.01
Quiz_Attempts                   0.08      0.03
Quiz_Score_Avg                  0.08      0.03

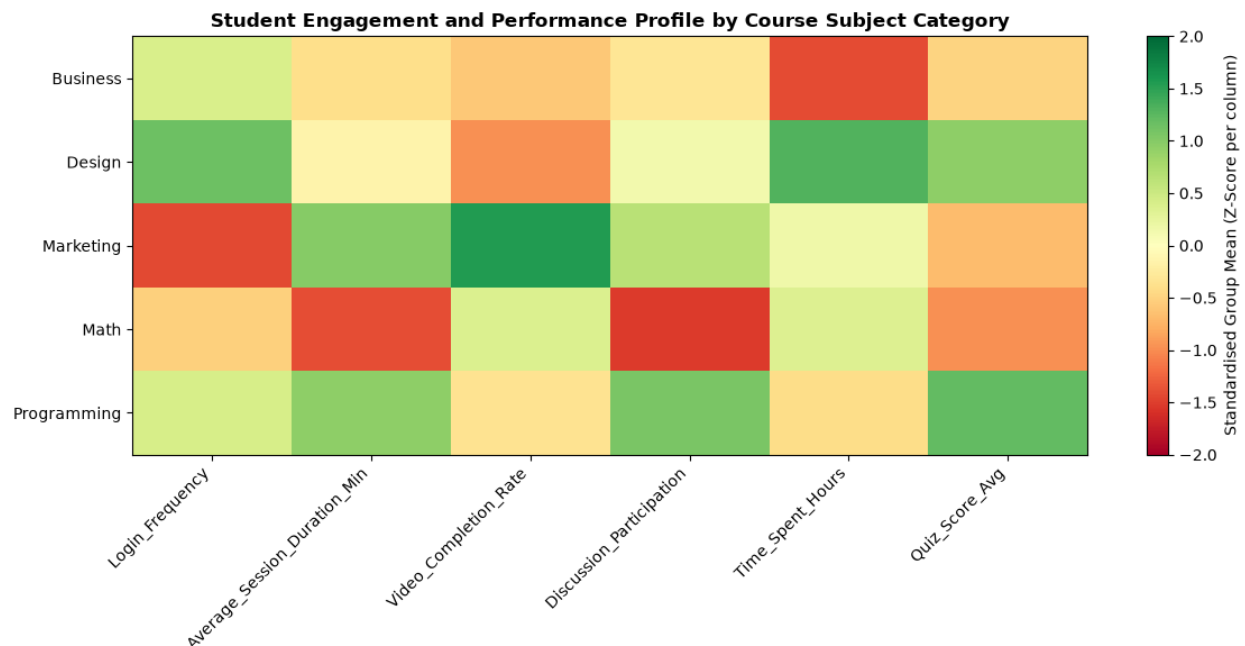
Days_Since_Last_Login  Quiz_Attempts  \
Progress_Percentage    -0.05      0.11
Course_Duration_Days     0.01      0.01
Average_Session_Duration_Min -0.03      0.08
Time_Spent_Hours        -0.01      0.03
Days_Since_Last_Login     1.00     -0.05
Quiz_Attempts            -0.05      1.00
Quiz_Score_Avg           -0.05      0.10

Quiz_Score_Avg
Progress_Percentage      0.13
Course_Duration_Days      0.01
Average_Session_Duration_Min  0.08
Time_Spent_Hours          0.03
Days_Since_Last_Login    -0.05
Quiz_Attempts             0.10
Quiz_Score_Avg            1.00

=====
--- Strongest Linear Association Detected ---
Strongest absolute correlation sits between: Average_Session_Duration_Min and Time_Spent_Hours
Correlation Value: 0.17
=====

```

The correlation heat map showing the 7 core student engagement variables doesn't show any strong associations between each variable. The highest linear association is between average session duration in minutes and time spent in hours; these have a correlation value of 0.17. This value isn't strong enough to suggest a clear association between the two, but it does suggest that someone who spends longer doing sessions may also spend more time working on the course.



This correlation heat map has some strong associations between student engagement and performance and the course subject the student is in.

Starting with business. Business has 1 strong association; time spent in hours. Time spent in hours has a strong negative correlation in business students, meaning that business students will spend less time on their course than other courses.

Second is design. Design has 3 strong positive correlations (login frequency, time spent in hours, and quiz score average) and 1 strong negative correlation (video completion rate). These strong correlations mean that design students will spend more time, login more frequently, and score higher on their quiz while completing less of their videos than other course.

Next is marketing. Marketing has 2 strong positive correlations (video completion rate and average session duration) and 1 strong negative correlation (login frequency). This means

marketing students will login less often but spend longer working once they login and complete more of their videos then other courses.

Fourth is math with 2 strong negative correlations (average session duration in minutes and discussion participation). These strong negative correlations, show that math students will spend less time per session and are less active in discussions than other courses.

Finally, is programming, with 3 strongly positive correlations (average session duration, discussion participation, and quiz score average). Programming students typically spend longer during a session, score higher in their quizzes, and are more active participants in discussions then other courses.

```
Students enrolled per Course Subject Category:
Category
Business      3830
Design        2552
Marketing      3756
Math           3057
Programming    16805
Name: count, dtype: int64
```

With a little over half the entire population, Programming has the highest number of students out of all the courses. This suggests that the above heat correlation map has a better accuracy for the programming category then the other courses. The other 4 categories have similar levels of students enrolled. This suggests that the above heat map could be less representative of student performance in these courses, as compared to the results from the programming course.

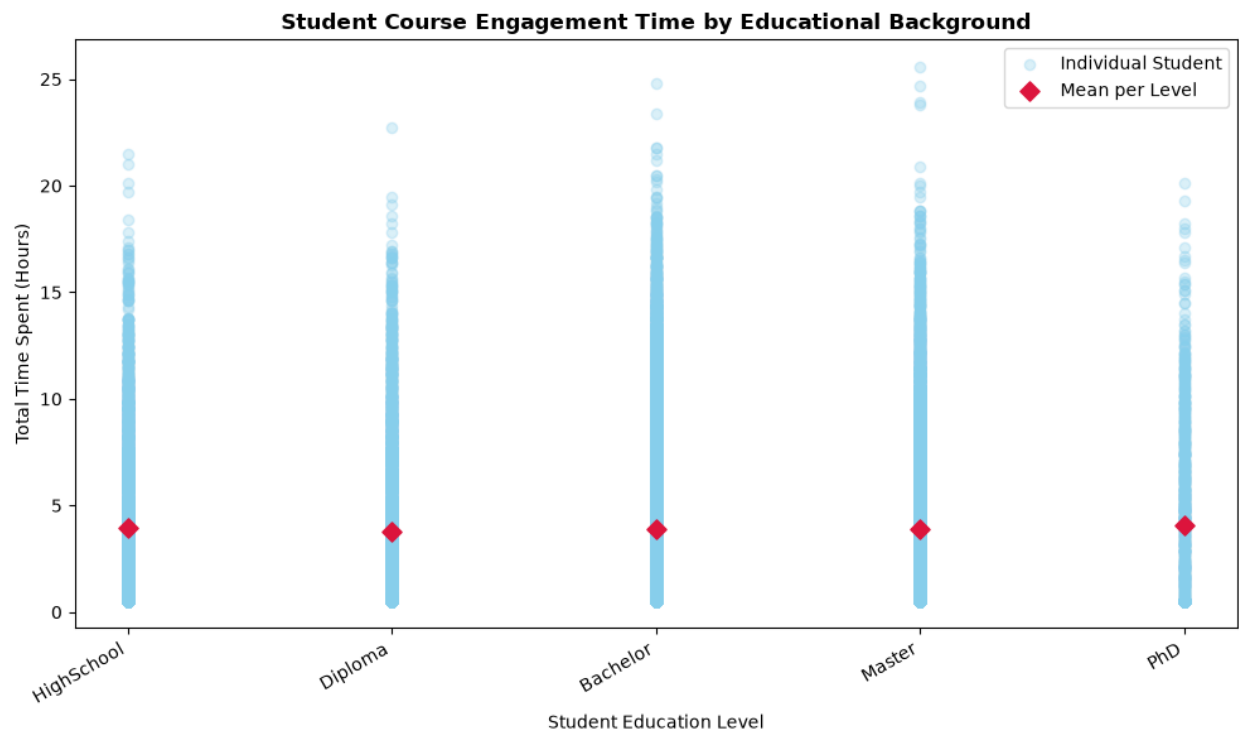
```
Mean engagement profile per Subject Category:
      Login_Frequency  Average_Session_Duration_Min \
Category
Business              4.8                        33.8
Design                4.8                        33.8
Marketing             4.7                        33.9
Math                  4.8                        33.7
Programming           4.8                        33.9

      Video_Completion_Rate  Discussion_Participation \
Category
Business                   62.2                      2.3
Design                     62.1                      2.3
Marketing                   62.7                      2.3
Math                       62.4                      2.3
Programming                 62.2                      2.3

      Time_Spent_Hours  Quiz_Score_Avg
Category
Business              3.8             73.0
Design                4.0             73.3
Marketing              3.9             73.0
Math                  3.9             72.9
Programming            3.9             73.3
```

The above capture shows the mean value for each course in each engagement category. All courses have very similar results with only small variation between the highest and lowest values in each category. For example, in average session duration, the difference

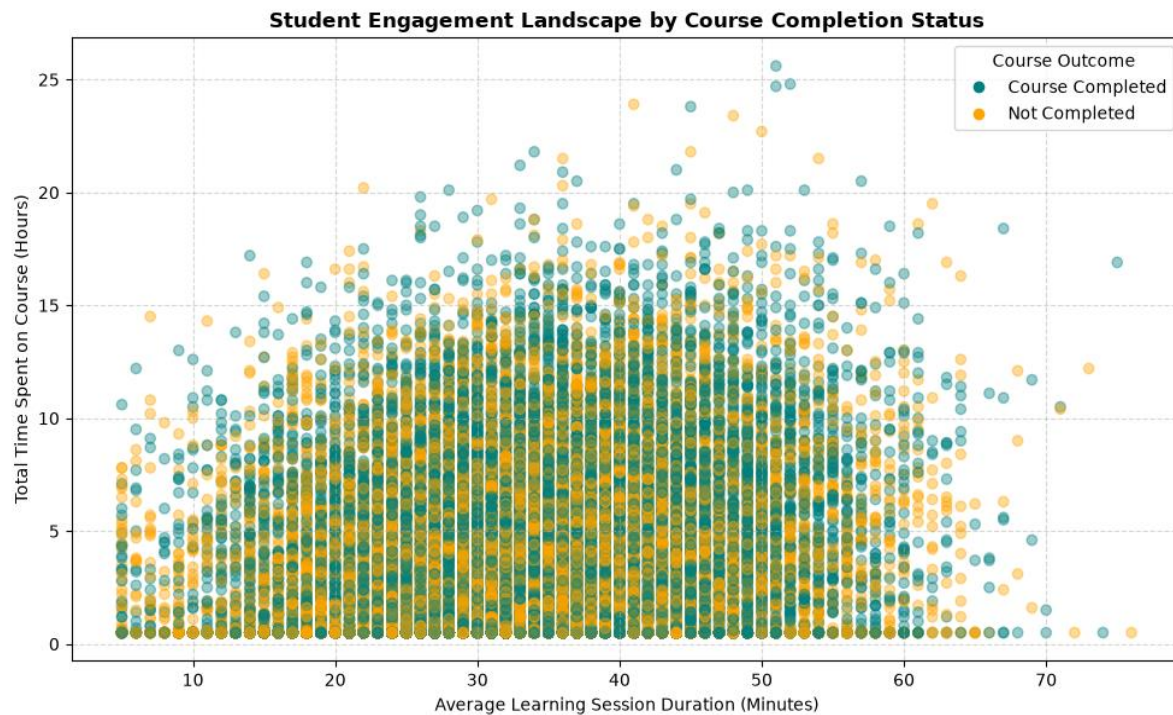
between the lowest (33.7), median (33.8), and highest (33.9) is only 0.1 minutes each. This suggests that the expected difference from a negative or positive correlation, on the student engagement and performance by course heat map, might mean that those course with strong correlations have a very small difference from the average across all courses.



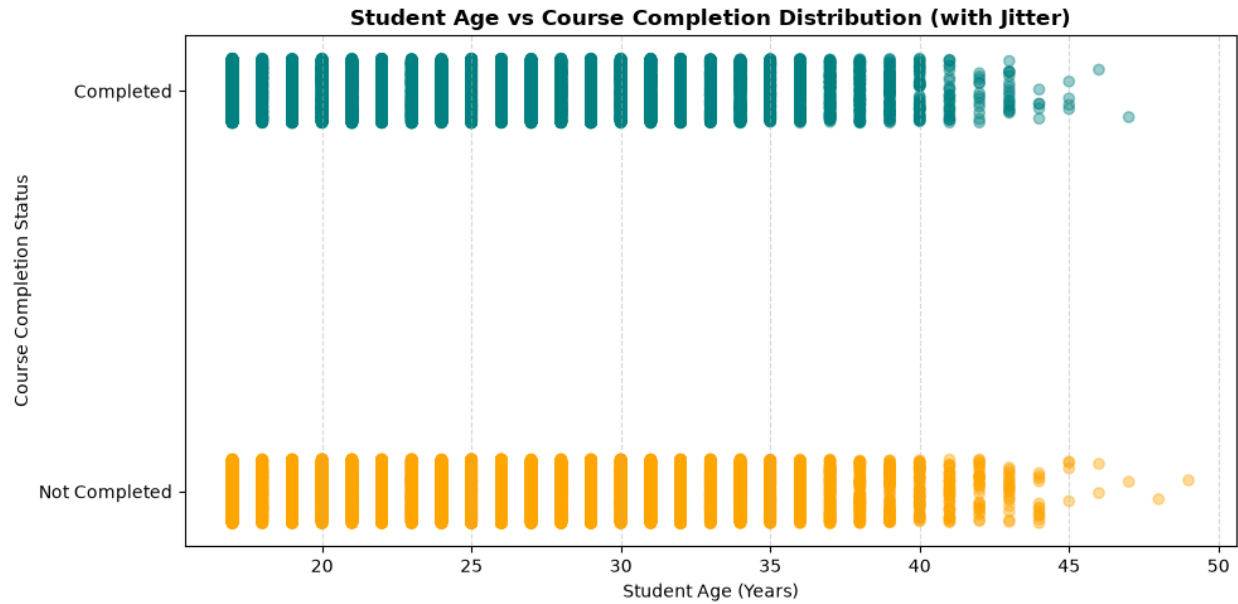
```
Mean Time Spent (Hours) by Education Level:  
Education_Level  
HighSchool    3.936485  
Diploma       3.798600  
Bachelor      3.876519  
Master        3.901152  
PhD           4.093866  
Name: Time_Spent_Hours, dtype: float64
```

This Graph showing course engagement time by educational background, has very similar results for each category of education level. Looking a little below the graph, the following capture lists the average time spent in hours and shows numerically the similarity between each category. The difference between the highest value (4.094) and the Lowest value (3.799) is 0.295 hours. All categories of education also show a very similar trend on the graph. They all have a very strong collection of values from just above 0 to around the 12-hours mark, where they start thinning out. With the mean time spent being at around the 4-

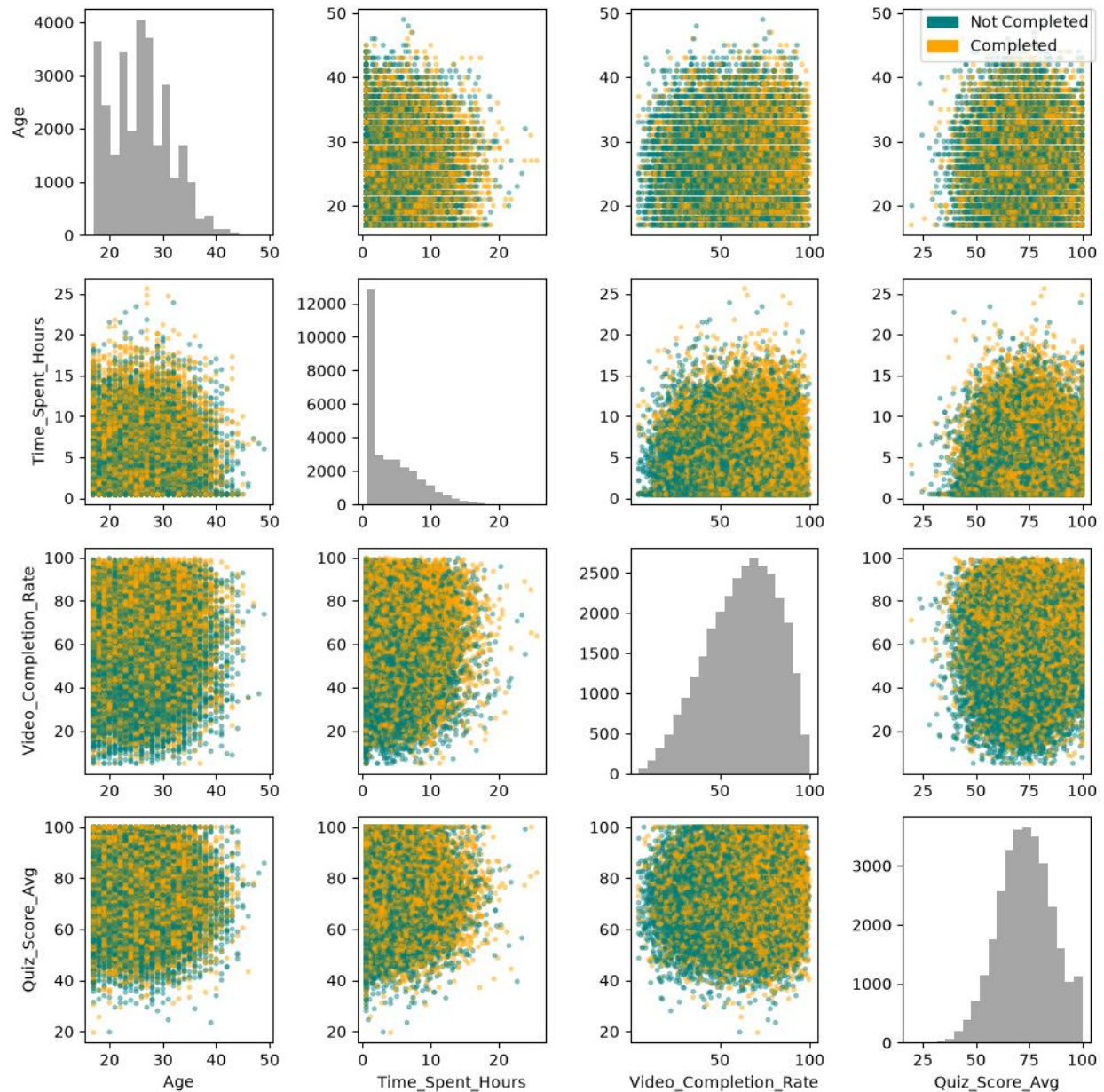
hour mark for each of the categories it points to most results being much closer to just above 0 then the 10- or 12-hours range.



In the graph above there is a singular central grouping of all results around 35 minutes. This group mainly spans from 10 minutes to 55 minutes and contains a very even mixture of course completed students and not completed students. Both groups also have an even number of outliers from this main bunching. There is no clear pattern between the course completed and not completed students in terms of total time spent and/or average session duration. There is a general trend of spending lower than 15 hours total on the course.



The age of the student does not seem to strongly influence whether they complete the course or not, as there is an equal distribution amongst most groups. Towards the end, however, more of the plots show older students past 45 having not completed their course, but there are too few counts to ascertain that this pattern is representative of the wider population.



The graphs on the diagonal are all number of results per value of the category. These show the general shape of the data and aren't indicative of completed or not completed course results. When the categories are compared against different categories then they show completed or not completed course results using colour. This allows for visual distinction between result. Although not many trends are strikingly obvious, the video completion rate category has an interesting gradient across each of the coloured graphs. There is a stronger presence of yellow (completed) towards the top of this category while the lower end of these graphs is more filled with blue (not completed). The other 3 categories have very even distributions across them of yellow and blue, with no other obvious trends being presented.


```
--- CONTINGENCY TABLE ---
```

Completed_Status	False	True
Gender		
Female	7567	7583
Male	7318	6917
Other	323	292

```
-----  
chi2 = 7.11, p = 0.028603
```

```
Reject H0: Gender and course completion are significantly associated in this sample.
```

With a chi-square score of 7.11, and a p of 0.286, there are significant gaps between the observed results, and the expected results if gender doesn't have any correlation. This is why the H0 (null hypothesis) is rejected. This means that there is a significant association between gender and course completion.

```
Mean Quiz Score (Completed): 74.27069361817199
```

```
Mean Quiz Score (Incomplete): 72.14109679116254
```

```
t = 14.71, p = 0.0000
```

```
Reject H0: The mean quiz scores between the two groups differ significantly.
```

Since the t score is 14.71 and the p score is less than 0.0000, there is a significant gap between the observed results, and the expected results if course completion doesn't have any correlation. This is why the H0 (null hypothesis) is rejected. This means that there is a significant correlation between completing the course and average quiz scores.